

BEFORE MEMORY

GNUSim8085 - 8085 Microprocessor Simulator

File Reset Assembler Debug Help

Registers

Register	Value	Flag
A	08	S 0
BC	05 00	Z 0
DE	00 00	AC 0
HL	00 00	P 0
PSW	00 00	C 0
PC	42 13	
SP	90 00	
Int-Reg	00	

Decimal - Hex Conversion

Decimal: 0 Hex: 0

To Hex To Dec

I/O Ports

0 - + 00

Update Port Value

Memory

0 - + 00

Update Memory

Load me at:

```

1 LXI SP,9000H
2 LDA 8050H
3 MOV B,A
4 LDA 8051H
5 ADD B
6 STA 8052H
7 JMP END
8 NOP
9 END: HLT

```

Start 8050h

Address (Hex)	Address	Data
8050	32848	9
8051	32849	3
8052	32850	0
8053	32851	0
8054	32852	0
8055	32853	0
8056	32854	0
8057	32855	0
8058	32856	0
8059	32857	0
805A	32858	0
805B	32859	0
805C	32860	0
805D	32861	0

Line No Assembler Message

0 Program assembled successfully

Simulator: Idle

AFTER MEMORY

GNUSim8085 - 8085 Microprocessor Simulator

File Reset Assembler Debug Help

Registers

Register	Value	Flag
A	0C	S 0
BC	09 00	Z 0
DE	00 00	AC 0
HL	00 00	P 1
PSW	00 00	C 0
PC	42 13	
SP	90 00	
Int-Reg	00	

Decimal - Hex Conversion

Decimal: 0 Hex: 0

To Hex To Dec

I/O Ports

0 - + 00

Update Port Value

Memory

0 - + 00

Update Memory

Load me at:

```

1 LXI SP,9000H
2 LDA 8050H
3 MOV B,A
4 LDA 8051H
5 ADD B
6 STA 8052H
7 JMP END
8 NOP
9 END: HLT

```

Start 8050h

Address (Hex)	Address	Data
8050	32848	9
8051	32849	3
8052	32850	12
8053	32851	0
8054	32852	0
8055	32853	0
8056	32854	0
8057	32855	0
8058	32856	0
8059	32857	0
805A	32858	0
805B	32859	0
805C	32860	0
805D	32861	0

Line No Assembler Message

0 Program assembled successfully

Simulator: Idle

Aim :- To Study the execution of data transfer, arithmetic and branch instruction in 8085 Microprocessor using an 8085 simulator.

Program :-

```
LDA 8050H
MOV B, A
LDA 8051H
ADD B
STA 8052H
JMP END
END : HLT
```

Execution cycle :-

Fetch → Decode → Execute.

observation Table :-

parameter	observation
Data 8050	09H
Data 8051	03H
Result 8052	00H 12H
program counter	changes after every instruction
Stack pointer	No change
Memory	8052 stores the sum.

②

Aim :- To perform bitwise operations (AND, OR, XOR, and NOT) on Hexadecimal values.

Procedure :-

→ Two Hexadecimal numbers

• A = 3CH → Binary :- 00111100

• B = A5H → A5 :- 10100101

Bitwise operations :-

operations	Results (Binary)	Result (Hex)
AND	00100100	24H
OR	10111101	BDH
XOR	10011001	99H
NOT (3C)	11000011	C3H

Result :- The bitwise operations were successfully performed using the windows calculator.

NOT Operator

Calculator - Programmer mode

NOT(3C)

FFFF FFFF FFFF FFC3

HEX: FFFF FFFF FFFF FFC3
DEC: -61
OCT: 1 777 777 777 777 777 703
BIN: 1111 1111 1111 1111 1111 1111 1111 1111
1111 1111 1111 1111 1111 1111 1100 0011

QWORD MS Mv

Bitwise Bit shift

A << >> CE ☒
B () % ÷
C 7 8 9 ×
D 4 5 6 -
E 1 2 3 +
F +/- 0 =

XOR Operator

Calculator - Programmer mode

3C XOR A5 =

99

HEX: 99
DEC: 153
OCT: 231
BIN: 1001 1001

QWORD MS Mv

Bitwise Bit shift

A << >> CE ☒
B () % ÷
C 7 8 9 ×
D 4 5 6 -
E 1 2 3 +
F +/- 0 . =

Calculator - Programmer mode

BD

HEX: BD
DEC: 189
OCT: 275
BIN: 1011 1101

QWORD MS Mv

Bitwise Bit shift

A << >> CE ☒
B () % ÷
C 7 8 9 ×
D 4 5 6 -
E 1 2 3 +
F +/- 0 =

OR Operator

Calculator - Programmer mode

3C AND A5 =

24

HEX: 24
DEC: 36
OCT: 44
BIN: 0010 0100

QWORD MS Mv

Bitwise Bit shift

A << >> CE ☒
B () % ÷
C 7 8 9 ×
D 4 5 6 -
E 1 2 3 +
F +/- 0 . =

AND Operator

cpu, memory and I/O

open :-

- Notepad
- Media player ✓
- google Chrome

component	Function
cpu	Executes instructions & processes data.
RAM	stores active programs temporarily
I/O Devices	keyboard, mouse, monitor
System BUS	Transfers data b/w cpu, memory and I/O devices.

Observation :-

- cpu usage increases while applications are running.
- RAM usage increases because multiple programs are loaded.
- I/O devices handle user input and display output.

Conclusion :-

cpu, memory and I/O devices work together to execute multiple applications efficiently.

MEDIA PLAYER- TASK MANAGER (PERFORMANCE)

